

6. Project Testing Phase

6.1 Introduction

Testing verifies that ComicCraft satisfies the specified requirements and performs its functions correctly. It identifies errors, security issues, integration problems, and unexpected behavior. Each module is tested individually and then as an integrated system.

6.2 Objectives of Testing

- To verify that all requirements are implemented correctly.
- To identify and correct software errors.
- To ensure authentication works properly and protected data is inaccessible to unauthorized users.
- To verify that comics, characters, panels, and images are stored and retrieved correctly.
- To evaluate the quality, relevance, and consistency of Gemini-generated scripts and artwork.
- To verify that export produces correct PDF and image files.
- To verify usability and reliability.

6.3 Testing Strategy

Unit Testing → Integration Testing → System Testing → User Acceptance Testing

6.4 Unit Testing

Individual functions are tested in isolation, including authentication functions, input validation, prompt-building functions, script JSON parsing, character description assembly, page layout calculations, bubble position handling, and database operations. For example, the number of pages is calculated as $\text{Pages} = \text{ceil}(\text{Total Panels} \div \text{Panels per Page})$. Different input combinations, including empty input, very long input, zero panels, and invalid JSON, are tested to confirm the expected result.

6.5 Integration Testing

- Frontend and backend communication.
- Backend and database operations.
- Authentication and protected APIs.
- Backend and Gemini text model (script generation).
- Backend and Gemini image model (panel generation).
- Panel images and bubble editor.
- Layout module and PDF/PNG export.

6.6 System Testing

Complete workflows are tested: registration, login, entering a story idea, generating and editing the script, defining characters, generating panel images, regenerating a panel, adding and editing speech bubbles, previewing the layout, exporting the comic, viewing the gallery, and logging out.

6.7 Functional Test Cases

Test ID	Function	Test Input / Action	Expected Result
FT01	Registration	Enter valid user details	Account is created
FT02	Registration	Enter incomplete details	Validation message appears
FT03	Login	Enter valid credentials	User is logged in
FT04	Login	Enter incorrect password	Login is rejected
FT05	Story Input	Submit an empty idea	Validation message appears
FT06	Script Generation	Enter idea, genre, and 6 panels	Structured script with 6 panels is generated
FT07	Script Editing	Edit dialogue of a panel and save	Updated text is stored
FT08	Characters	Add a character with description	Character is saved and used in panel prompts
FT09	Panel Generation	Generate panels for a script	An image is created for each panel in the chosen style
FT10	Consistency	Compare a character across panels	Appearance is reasonably consistent
FT11	Regeneration	Regenerate a single panel	Only that panel image is replaced
FT12	Bubble Editor	Move and resize a speech bubble	Position and size are saved
FT13	Layout	Select a 4-panel page layout	Panels are arranged correctly into pages
FT14	Export	Export as PDF	A correct, readable PDF is downloaded
FT15	Gallery	Open gallery	Saved comics are displayed
FT16	Logout	Select logout	Session ends

6.8 Authentication and Security Testing

- Access to protected APIs without a token is rejected.
- Expired or invalid tokens are rejected.
- A user cannot view or edit another user's comics or images.
- Passwords are stored as hashes.
- The Gemini API key is not exposed to the browser.

6.9 AI Response Testing

Gemini outputs are evaluated for relevance, story coherence, panel pacing, format validity, visual quality, style consistency, and character consistency using sample scenarios such as a children's adventure, a superhero origin, a school-life comedy, and a science-fiction mystery. Cases tested

include empty or very long ideas, vague prompts, prompts that trigger safety blocks, and malformed JSON output. Because AI-generated images can occasionally contain anatomical errors or inconsistencies, users can regenerate any panel. The application also displays a notice that content is AI-generated and that users are responsible for how they use and share it.

6.10 Performance and Usability Testing

Page load time, script generation time, per-panel image generation time, and export time are observed, along with behavior during Gemini delays. Usability is checked for navigation, clarity of messages, bubble editing ease, and responsiveness on desktop and mobile screens.

6.11 User Acceptance Testing

A small group of students, hobby writers, and teachers uses the application and provides feedback on ease of use, story quality, artwork quality, and the editing tools. Reported issues are corrected before the final demonstration.

6.12 Testing Summary

After testing and correction of defects, all major modules were found to operate according to the requirements, and the application was considered ready for demonstration.